

ABC450 G - Random Subtraction 题解

题意概括

给定一个长度为 N 的序列。每次等概率选择一对有序下标 (i, j) ，删除这两个位置上的数，再把 $A_i - A_j$ 放回序列末尾。重复到只剩一个数 x ，要求 x^2 的期望值。

解题思路

直接追踪最终那个数很难，但 x^2 可以通过两个整体量来描述。

设当前序列长度为 k ，当前数列记为 $b_1, b_2, \dots, b_k$ 。定义：

- $S = b_1^2 + b_2^2 + \dots + b_k^2$
- $T = (b_1 + b_2 + \dots + b_k)^2$

最后只剩一个数时，显然有 $x^2 = S = T$ 。所以只要研究每次操作后 S 和 T 的期望如何变化即可。

一、推导 S 的转移

若本次选中的是 b_i 和 b_j ，新加入的数是 $b_i - b_j$ ，那么：

- $S' = S - b_i^2 - b_j^2 + (b_i - b_j)^2 = S - 2 b_i b_j$

因为 (i, j) 是有序且等概率地从所有不同下标中选出的，所以

- $\sum_{i \neq j} b_i b_j = \left(\sum b_i\right)^2 - \sum b_i^2 = T - S$

一共有 $k(k-1)$ 个有序对，因此

- $E[S'] = S - \frac{2(T - S)}{k(k-1)}$

二、推导 T 的转移

操作后总和会变成：

- $(b_1 + \dots + b_k) - b_i - b_j + (b_i - b_j) = (b_1 + \dots + b_k) - 2 b_j$

也就是说，总和只和第二个被选中的位置有关。于是

- $T' = \left(\sum b_i - 2 b_j\right)^2$

对所有 j 取平均即可：

- $E[T'] = \frac{4}{k} S + \frac{k-4}{k} T$

三、改写成更容易递推的形式

记当前长度为 k 时:

- $s_k = E[S]$
- $t_k = E[T]$
- $d_k = t_k - s_k$

把上面两个式子相减, 可以得到:

- $d_{k-1} = \frac{(k-2)(k-3)}{k(k-1)} d_k$

再把 $t_k = s_k + d_k$ 代回 s_k 的转移式, 可得:

- $s_{k-1} = s_k - \frac{2 d_k}{k(k-1)}$

现在有两个很重要的结论:

1. 当 $k = 3$ 时, 系数变成 0, 所以 $d_2 = 0$;
2. 也就是说, 只要 $N \geq 3$, 做到长度 2 时就一定满足 $t_2 = s_2$, 最后一步不会再改变 x^2 的期望。

把 d_k 连乘展开, 可以得到:

- $d_k = \frac{k(k-1)^2(k-2)}{N(N-1)^2(N-2)} d_N$, 其中 $3 \leq k \leq N$

再把它代回 $s_{k-1} = s_k - \frac{2 d_k}{k(k-1)}$, 从 $k = N$ 一路累加到 $k = 3$, 可得

- $s_1 = s_N - \frac{2 d_N}{3(N-1)}$

而初始时:

- $s_N = \sum A_i^2$
- $t_N = \left(\sum A_i\right)^2$
- $d_N = t_N - s_N$

于是当 $N \geq 3$ 时,

- $s_1 = \frac{(3N-1)\sum A_i^2 - 2\left(\sum A_i\right)^2}{3(N-1)}$

这就是答案。

边界单独处理:

- $N = 1$ 时, 答案就是 A_1^2
- $N = 2$ 时, 最后一定得到 $A_1 - A_2$ 或 $A_2 - A_1$, 平方相同, 所以答案是 $(A_1 - A_2)^2$

因此整题只需要求出 $\sum A_i$ 和 $\sum A_i^2$ 即可，时间复杂度为 $O(N)$ 。

正确性说明

先说明两个递推式正确。

对任意一次操作，新的平方和满足 $S' = S - 2 b_i b_j$ 。而所有有序对的 $b_i b_j$ 总和恰好是 $T - S$ ，平均以后就得到

$$\bullet E[S'] = S - \frac{2(T - S)}{k(k - 1)}$$

同理，新的总和是 $\sum b_i - 2 b_j$ ，只与第二个被选中的元素有关。把这个平方对所有 j 平均，就得到

$$\bullet E[T'] = \frac{4}{k} S + \frac{k - 4}{k} T$$

因此 s_k 与 t_k 的递推成立。

再看 $d_k = t_k - s_k$ 。把上面两个递推相减，可以直接化简出

$$\bullet d_{k - 1} = \frac{(k - 2)(k - 3)}{k(k - 1)} d_k$$

所以 d_k 的连乘表达式也成立。当 $k = 3$ 时，这个系数为 0，于是 $d_2 = 0$ 。这说明长度降到 2 以后， T 与 S 的期望已经相等；而长度 2 再做一步后只会剩下一个数，此时答案正是 S 。

最后，由 $s_{k - 1} = s_k - \frac{2 d_k}{k(k - 1)}$ 把从 N 到 3 的所有贡献累加起来，就唯一确定了 s_1 。它正是最终 x^2 的期望，所以公式正确。

复杂度

- 时间复杂度： $O(N)$
- 空间复杂度： $O(1)$

参考实现

```
#include<bits/stdc++.h>
using namespace std;

const int mod = 998244353;
const int maxn = 200000;

int n;
long long a[maxn + 5];
```

```
long long qpow(long long a, long long b){
    long long res = 1;
    while(b > 0){
        if(b & 1){
            res = res * a % mod;
        }
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

int main(){
    cin >> n;
    for(int i=1; i<=n; i++){
        cin >> a[i];
    }

    // 先求初始的 sum = \sum A_i, 以及 square_sum = \sum A_i^2
    long long sum = 0;
    long long square_sum = 0;
    for(int i=1; i<=n; i++){
        sum = (sum + a[i]) % mod;
        square_sum = (square_sum + a[i] * a[i]) % mod;
    }

    // N = 1 时, 最终只剩 A_1 本身
    if(n == 1){
        cout << square_sum % mod << '\n';
        return 0;
    }

    // N = 2 时, 最后一定得到 A_1 - A_2 或 A_2 - A_1, 平方相同
    if(n == 2){
        long long diff = (a[1] - a[2]) % mod;
        if(diff < 0){
            diff += mod;
        }
        cout << diff * diff % mod << '\n';
        return 0;
    }
}
```

```
// N >= 3 时, 直接套用
// ans = ((3N - 1) * \sum A_i^2 - 2 * (\sum A_i)^2) / (3(N - 1))
long long up = (3LL * n - 1) % mod * square_sum % mod;
up = (up - 2LL * sum % mod * sum % mod + mod) % mod;

long long down = 3LL * (n - 1) % mod;
long long ans = up * qpow(down, mod - 2) % mod;

cout << ans << '\n';

return 0;
}
```